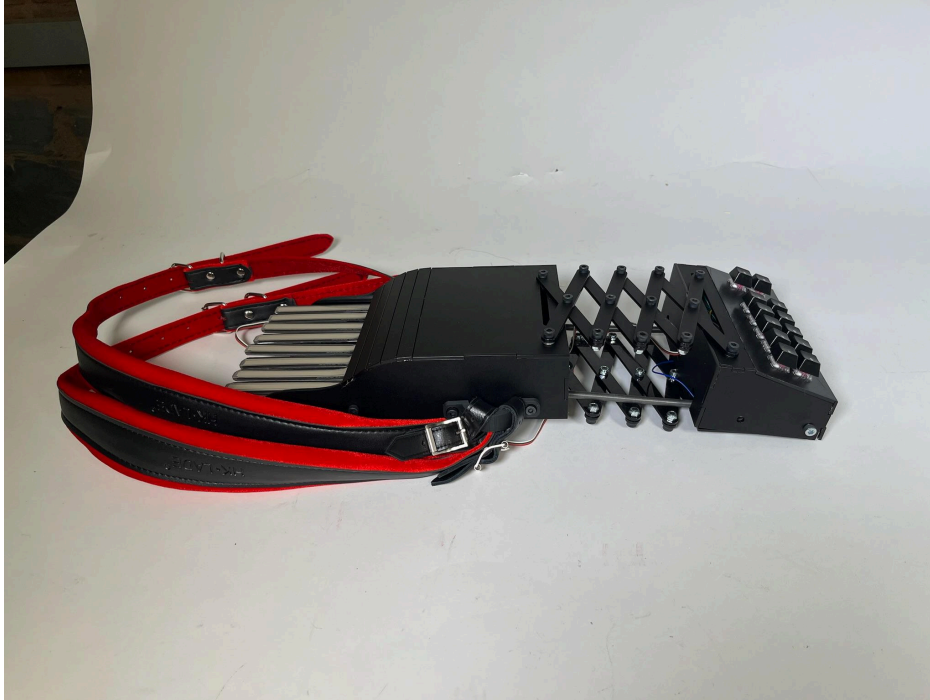


The CHORDIAN (v2)



Summary

The Chordian is an electronic “evolution” of an accordion. Just like a normal accordion, the player has access to multiple forms of unique expression. Our group strived to create an instrument that could be performed by itself, with multiple options for chords and singular notes. Additionally, the Chordian uses multiple types of sensors, each with different parameters to control. Overall, the Chordian successfully pairs the user’s multi-dimensional input with dynamic and musical generation.

For our final iteration of the instrument we wanted to achieve three main things:

1. Wireless capabilities
2. Secondary sensor implementation (in addition to our soft potentiometers)
3. Note and scale adjustment

With these new features, the instrument would be more intuitive and easier to interact with, thus generating more interesting sounds and music.

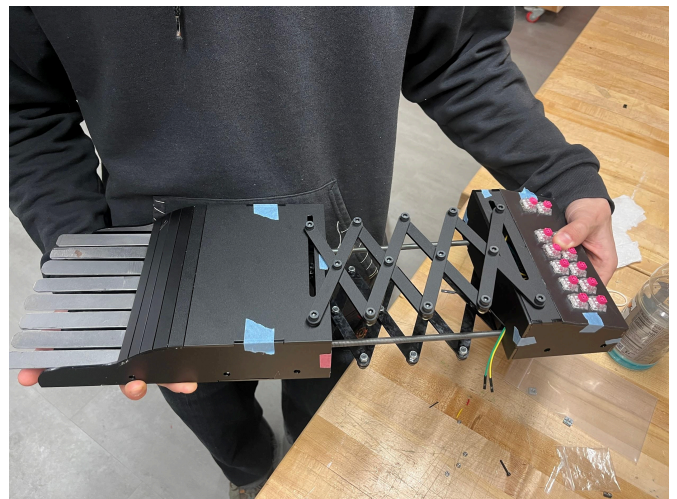
Instrument Layout and Playing

The Chordian is broken down into three main parts: the key box, the button box, and the linkage. The key and button boxes are connected by the linkage system to create a fully functioning accordion motion. Additionally, there are accordion straps connected to the key box to allow for further stability and playing ease.



Materials and Construction

The key box is composed of 8 metal keys, made out of steel sheet metal. Each key was cut and sized using the jump shear and the drill press was used to create the holes. The horizontal band sander was used to round the edges of the keys to create a more comfortable playing experience. On each key is one linear soft potentiometer and one force sensing resistor. On the outside of the box, there is a time of flight distance sensor attached. The sensors are soldered onto the protoboard that lives within the key box. The actual box is made out of laser cut acrylic, 3mm thick, small 3D printed parts for wiring organization and mounts, and aluminum extrusions. These extrusions provide structural support for metal keys and add some weight to improve playability and feel.

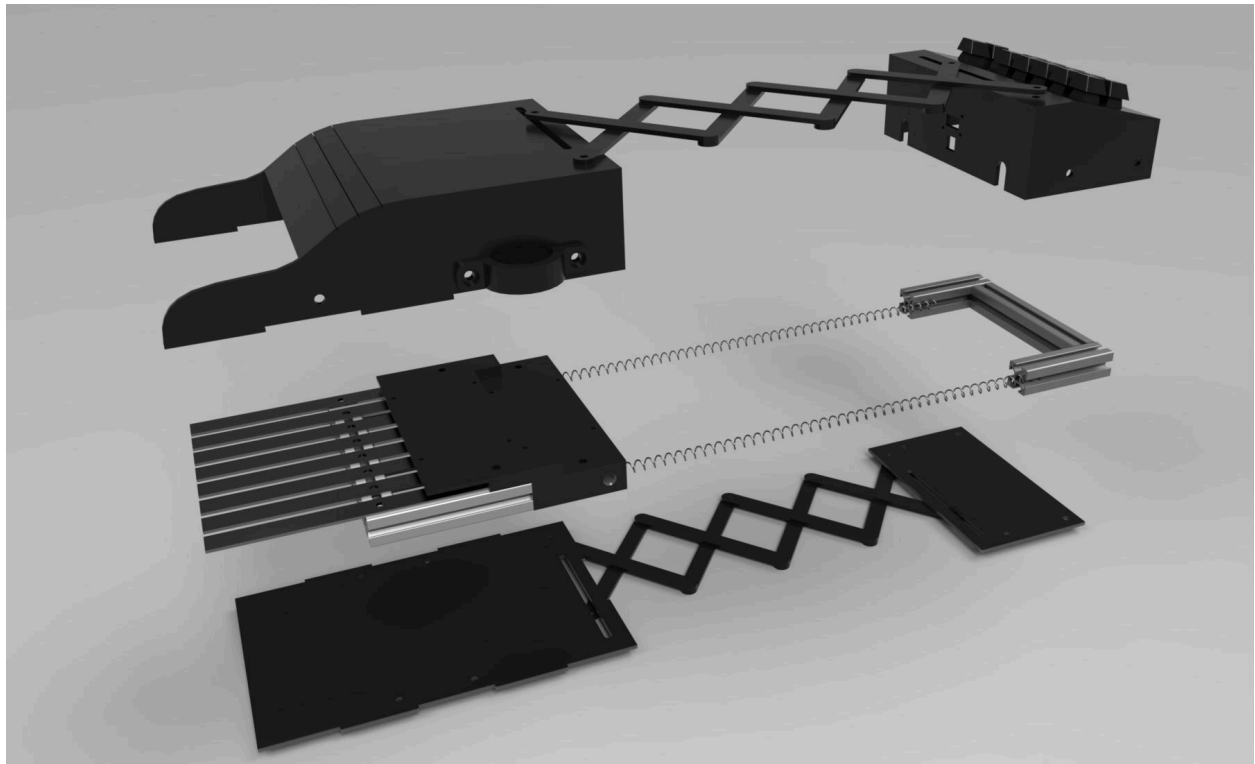


The linkage attaching the two boxes is also made out of laser cut acrylic, but also consists of multiple bolts, nuts, and washers to ensure smoothness when moving the boxes. There are also

two springs attached to the key and button boxes to produce a force that the user can resist, emulating the feel of a regular accordion.

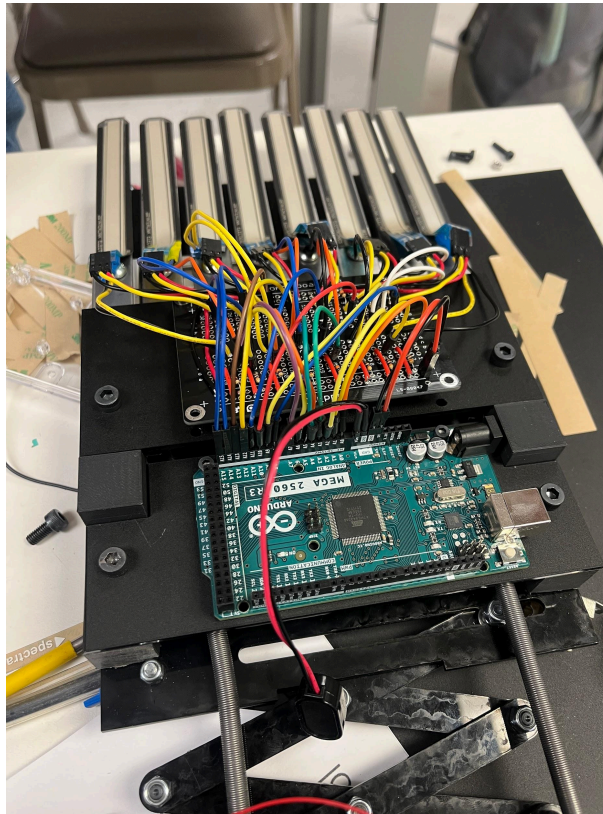
The button box, made out of laser cut acrylic, consists of 12 buttons (key switches) and 2 rotary encoders. There is a 3D printed sheet for the switches to click into place. The button box also uses aluminum extrusions to attach the different laser cut pieces with nuts and bolts, allowing for easy access if needed.

All of the laser cut pieces were first designed in OnShape, where we took advantage of designing with variables that we could easily change to quickly update our design choices.



Sensors and Circuitry

To read our 8 soft potentiometers, 8 force sensors, 1 time of flight sensor, 12 buttons, and 2 rotary encoders, we needed *three* microcontrollers.



Three controllers were necessary because when using the ESP-NOW protocol, it disables many of the analog pins on the ESP, which are crucial in our case. To bypass this, we use the Arduino Mega to first read in the soft potentiometers and force sensors. Using the SDA and SCL pins, we then send this data over I2C to the ESP32 in the button box. At the same time, the ESP in the button box reads the data from the button switches and the rotary encoders. After packaging up the information, the ESP can finally send to a *second* ESP which is plugged into the desktop computer that has the Max patch loaded up.

1. Arduino Mega

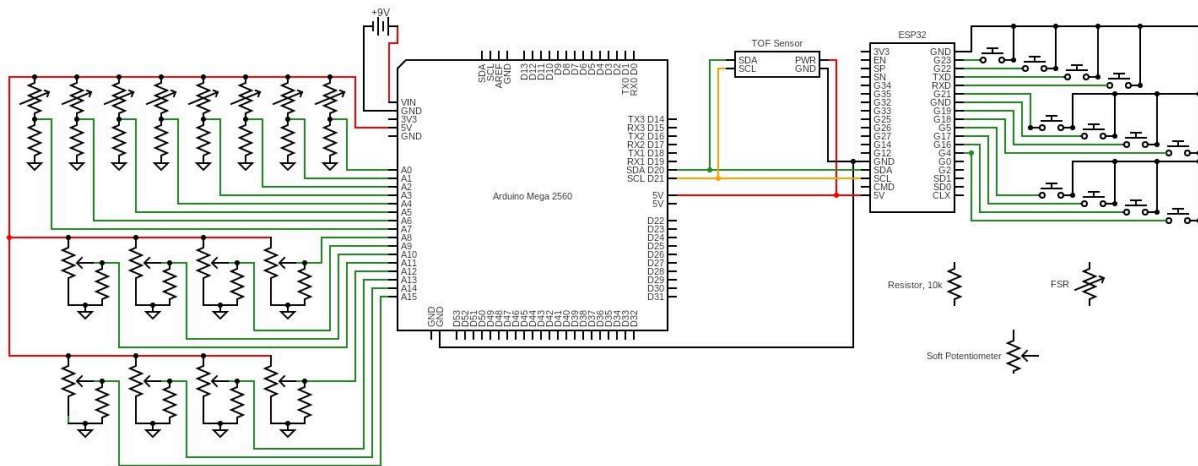
- a. For the soft potentiometers, there are three pins: one receiving 5 volts, one into ground, and the last one being sent to an analog pin and also being pulled down to ground by a 10 K ohm resistor. By wiring it this way, we create a voltage divider and decrease the noise the potentiometer produces.
- b. The force sensors are also very similar, although only having two pins: one receiving 5 volts and the other going into an analog pin and also a pulldown resistor.

2. ESP32 WROOM UE (1)

- The time of flight distance sensor uses the SDA and SCL pins and needs 5V and ground.
- The buttons do not act as a resistor, so we used the ESP's built in pull up resistors within the digital pins. Each button goes to a different digital pin and to ground.
- Each rotary encoder uses three digital pins: two to sense which direction the knob turns and one to sense when the button is pressed.

3. ESP32 WROOM UE (2)

- This microcontroller is not connected to any sensors, instead it acts as the “hub” microcontroller. Using the “ESP-NOW” protocol, we can communicate with our ESP32's over WiFi with decreased latency. All of the sensor data can be easily transferred from the hub ESP into the MAX patch to then complete our computational logic.

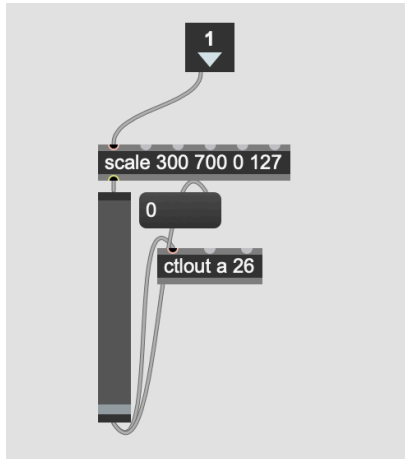


To better organize and solidify our wiring, we soldered all of the sensors and connections to two protoboards, except for the wires going into the arduino/ESP (analog pins, digital pins, 5V, GND). We also have a long cable running between the two boxes that transfers 5V, GND, SDA, and SCL.

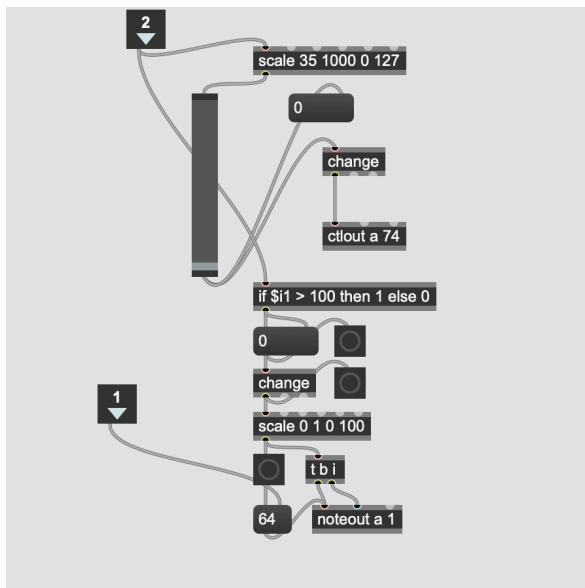
Max and Reason

When the sensor values are read by the hub ESP32, the values are then printed into the serial monitor. Whatever is in the serial monitor is then sent into our Max patch. By unpacking this list of numbers, we can control what our different sensors do.

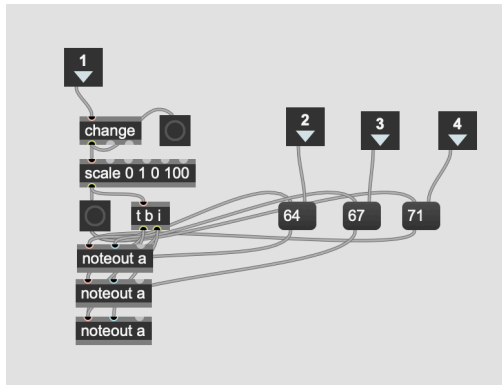
From pin A0 to A7, the force sensors are scaled to provide values from 0 to 127. With the force sensors we are changing the value of controller 26.



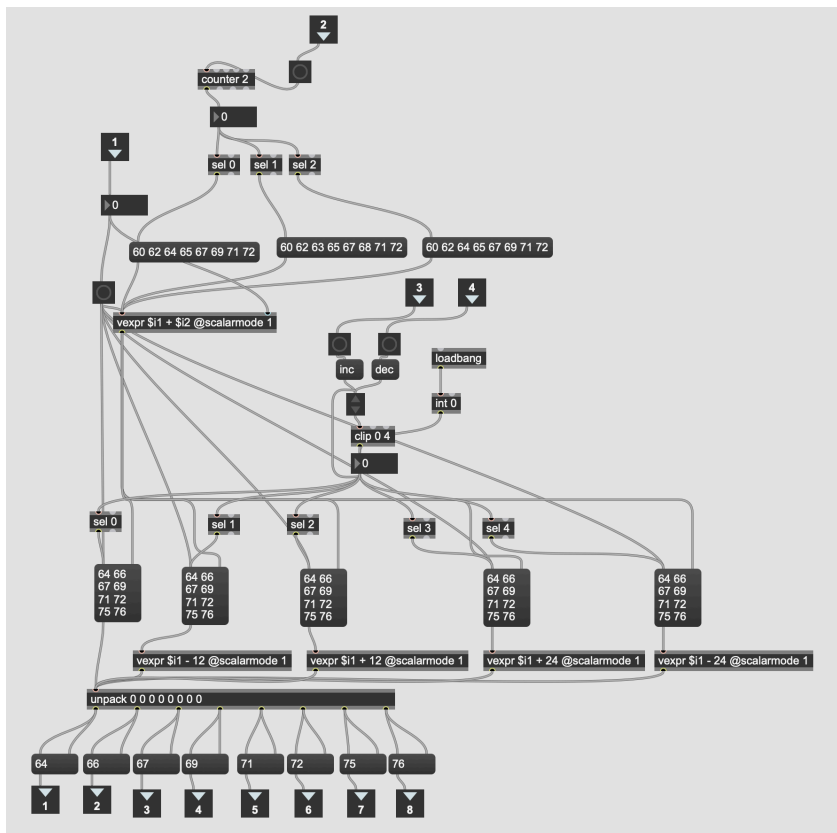
From pins A8 to A15, we handle the logic to play notes on our soft potentiometers. First, the values are scaled to fit the range of 0-127. Throughout the soft pot logic, we heavily rely on the *change* function. This will send a bang if the value is different from before. This way, we can send a note on when the soft potentiometer reads a value. Additionally, the Max patch allows for the soft potentiometer to send values to controller 74 (“Brightness”/filter frequency) depending on where your finger is on the potentiometer.



From pins D2 to D10, we handle the logic of the button presses. If a button is pressed down, then a chord (a series of simultaneous notes on messages) is sent to reason. For each button, a different chord.



The rotary encoders require more complicated arithmetic and logic to make changes to an entire list of values. When the knob is turned, it should increase or decrease the values by a certain amount.



Lastly, the distance sensor controls the volume of the instrument, but emulates the bellows of an accordion. By filtering the data in the code, the user can move the boxes to “generate” sound. The volume is based on the velocity between the two separate boxes.

Problems / Future Notes

Another issue that we ran into during the first iteration was sending the key box and the button box to two separate midi channels. We still ran into this issue this iteration. Although it seems to be simple on Max, sometimes notes on messages were sent to both channels, introducing some interference and odd sounding notes.

If I were to keep on working on this project, I would continue researching best methods for wireless capabilities and communication, as the three different microcontrollers introduce a lot of variables, increasing the points of failures. I would try to reduce the number of microcontrollers and also try to get rid of the “hub” controller and instead send data straight to Max.

Overall, I am extremely proud of this project and myself. I learned so much about microcontrollers, wireless communication, sensor types, reading and sending data, mechanical design, designing for ergonomics, and how to think more deeply about instruments and how we make music. This was so fun, interesting, and I can't wait to implement my new skills on new projects!